

Spring 2026: Programming for chemistry (in Python!)

Davide Ceresoli (davide.ceresoli@cnr.it)

Istituto di Scienze e Tecnologie Chimiche “G. Natta” (CNR-SCITEC) and
Dept. of Chemistry, University of Milan

September 28, 2026

- 1 Course logistics
- 2 The way to program
- 3 The Python programming language
- 4 Recap

- Dr. Davide Ceresoli, Senior Researcher at CNR-SCITEC
- email: davide.ceresoli@cnr.it, davide.ceresoli@unimi.it
- background: *laurea* in Materials Science, PhD in Physics
- activity: ab-initio DFT calculations, molecular dynamics, high-pressure, thermoelectrics, code development (Quantum-Espresso)

- 6 CFU, 48 hours (24 lectures two hours each)
- Computer room 311, settore didattico via Celoria:
 - tuesdays 08:30–10:30
 - fridays 08:30–10:30
- MyAriel course website (announcements, slides, ...):
<https://myariel.unimi.it/course/view.php?id=12407>
- Mirror course website (calendar, slides, ...):
<https://dceresoli.github.io/2026-Programming>

- October: 2, 6, 9, 13, 16, 20, 23, 27, 30
- November: 6, 10, 13, 17, 20, 24, 27
- December: 1, 4, 11, 15, 18, 22(??)
- January 2027: 8, 19(??)
- Course plan is subject to changes
- ATM strike Fridays...

- No previous knowledge of programming is assumed!
- By the end of the course, you will:
 - Understand fundamental concepts of programming imperative languages
 - Design algorithms to solve simple problems
 - Learn the Python programming language
 - Solve some chemistry-related problems
 - Have fun programming, maybe a computer game...

- There are plenty of online free Python books, tutorials and resources
- P. Wentworth *et al.*, *How to Think Like a Computer Scientist in Python 3*, available free at:
<https://openbookproject.net/thinkcs/python/english3e/>
- The official Python 3 documentation:
<https://docs.python.org/3/>
- For the exam: B. Stephenson, *The Python Workbook*, Springer

- We'll use the interactive **Jupyter** notebook most of the time. Here are the tools you need to install:
- Windows: Anaconda (*too much user friendly!*)
- Windows: WSL with Debian/Ubuntu
`sudo apt install python3 jupyter numpy scipy matplotlib`
- Linux and VSCode (*the only Microsoft-product that works!*)
- Mac OS: I have no experience!
- ...

Please

Deactivate any AI-assistant in Jupyter and VSCode!

- In the cloud: no installation, only need a browser, you get CPU (and GPU) for free
 - 1 Google Colab <https://colab.research.google.com/>
 - 2 Kaggle <https://www.kaggle.com/>
 - 3 Binder <https://mybinder.org/v2/gh/dceresoli/2026-Programming/HEAD>
 - 4 Microsoft Azure Notebooks, Datalore, ...

- Lectures: will be interactive, with questions and interactive problem solving
- Attendance is recommended
- **Please, do not use AI assistants such as: ChatGPT, Claude, Gemini!**
- There will be “free programming practice” days to catch up, exercise, propose problems
- Final exam: individual, solving 3-4 exercises from *The Python Workbook*, couple of general questions

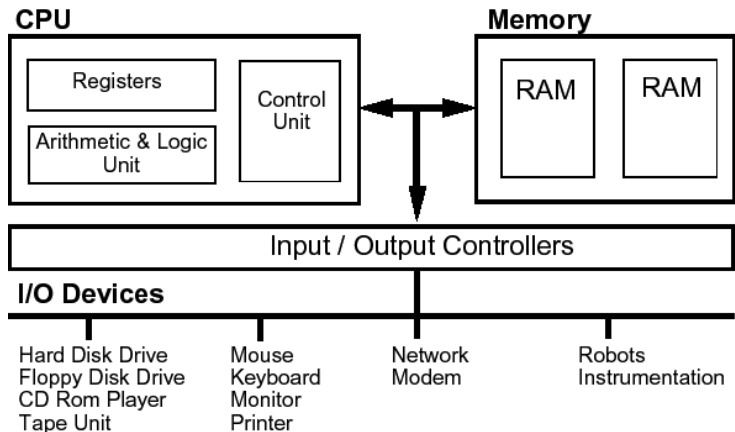
Questions?

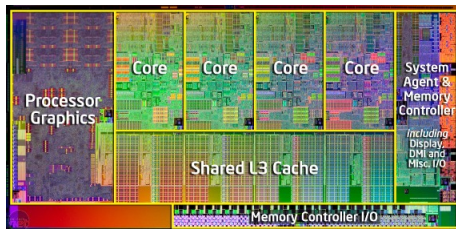


Copyright © Randy Glasbergen. www.glasbergen.com

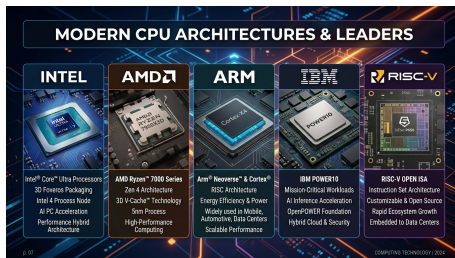
Take-home message

Computer don't solve problems (yet), people do!





- The CPU reads the both code and data from RAM
- The code is executed one instruction at the time, data is moved to/from memory
- Conditionally, the CPU jumps to a different location in the code
- The operating system (OS) takes care of:
 - scheduling execution of N processes on m CPU cores
 - move data to/from disk, network and RAM
 - interact with the hardware and with the user



- Intel and AMD implement the so called *x86-64* instruction set
- ARM CPUs are popular on smartphones and Apple devices
- IBM Power CPU is mostly used on mainframes
- small-power CPU architectures: RiscV, ...

The CPU understands only low-level *Assembly* instructions

Disassembly of section `.text`:

```
0000000000000000 <distance>:
0: 55                push    %rbp
1: 48 89 e5          mov     %rsp,%rbp
4: 48 83 ec 20       sub     $0x20,%rsp
8: f2 0f 11 45 f8    movsd   %xmm0,-0x8(%rbp)
d: f2 0f 11 4d f0    movsd   %xmm1,-0x10(%rbp)
12: f2 0f 11 55 e8    movsd   %xmm2,-0x18(%rbp)
17: f2 0f 10 45 f8    movsd   -0x8(%rbp),%xmm0
1c: 66 0f 28 c8       movapd  %xmm0,%xmm1
20: f2 0f 59 4d f8    mulsd   -0x8(%rbp),%xmm1
25: f2 0f 10 45 f0    movsd   -0x10(%rbp),%xmm0
2a: f2 0f 59 45 f0    mulsd   -0x10(%rbp),%xmm0
2f: f2 0f 58 c8       addsd   %xmm0,%xmm1
33: f2 0f 10 45 e8    movsd   -0x18(%rbp),%xmm0
38: f2 0f 59 45 e8    mulsd   -0x18(%rbp),%xmm0
3d: f2 0f 58 c1       addsd   %xmm1,%xmm0
41: e8 00 00 00 00    callq   46 <distance+0x46>
46: f2 0f 11 45 e0    movsd   %xmm0,-0x20(%rbp)
4b: 48 8b 45 e0       mov     -0x20(%rbp),%rax
4f: 48 89 45 e0       mov     %rax,-0x20(%rbp)
53: f2 0f 10 45 e0    movsd   -0x20(%rbp),%xmm0
58: c9               leaveq  %rax,%rbp
59: c3               retq
```

- These are very simple instructions such as:
 - move bytes from RAM to CPU registers and back
 - do arithmetic operations on CPU registers
 - advance to the next instruction, or jump to another location
 - each CPU has a different set of instructions!

For us, it is easier to program in higher-level languages

The previous block of bytes and assembly instructions, is equivalent to:

```
#include <math.h>

double distance(double x, double y, double z)
{
    return sqrt(x*x + y*y + z*z);
}
```

This code runs on almost every CPU

This is why we need to learn programming languages!

Compiled languages

- Source code is analyzed entirely and checked for errors
- Then, the source code is *compiled* (aka translated) into assembly instructions
- Possibility to perform deep code optimizations
- Finally, multiple code units are *linked* together and with *libraries* of functions
- The results is the *executable*, that can be run by the operating system *at the maximum performance*
- Compiled languages: C, C++, Fortran, Pascal, Go, Rust, ...

Interpreted languages

- Source code is processed line-by-line and checked for errors
- The interpreter *runs* the source code, also interactively
- Possibility to debug and inspect the code while writing
- Usually very easy and fun to program with
- Difficult to optimize, performance much lower than compiled languages
- Interpreted languages: Python, Basic, Forth, Javascript, PHP, Lisp, Octave, ...

In the middle: just-in-time compiled languages

- Source code is processed line-by-line and checked for errors, also interactively
- The code is compiled *on-the-fly* to CPU instructions
- Performance is between compiled and interpreted languages
- Price to pay is longer startup time and need of support run-time libraries
- JIT languages: Java, Javascript, Julia, ...

Programming language popularity

Sep 2026	Sep 2025	Change	Programming Language		Ratings	Change
1	1			Python	17.76%	-8.22%
2	3	▲		C	10.28%	+1.63%
3	2	▼		C++	8.67%	-0.13%
4	4			Java	7.54%	-0.81%
5	5			C#	4.22%	-2.16%
6	6			JavaScript	2.76%	-0.46%
7	7			Visual Basic	2.55%	-0.28%
8	11	▲		SQL	2.16%	+0.29%
9	13	▲▲		R	1.69%	+0.27%
10	18	▲▲		Rust	1.34%	+0.33%
11	12	▲		Fortran	1.24%	-0.25%
12	8	▼▼		Go	1.10%	-1.22%
13	9	▼▼		Delphi/Object Pascal	1.08%	-1.18%
14	15	▲		PHP	1.04%	-0.21%
15	16	▲		Scratch	0.99%	-0.19%
16	17	▲		Assembly language	0.89%	-0.15%

Python is slow!

- Can be 100x times slower than C/C++/Fortran

Python is fast!

- If you use C/C++/Fortran libraries

- Conceived by Guido van Rossum in the '90s (Python 1)
- Python 2 was popular until few years ago, not used anymore
- We'll be using at least Python 3.11 (shipped with Linux Debian 12)
- Python 2 and Python 3 are incompatible, the code must be converted
- *I will point out major differences between Python and other programming languages*

How does Python look like?

```
#!/usr/bin/env python
# Anderson model in 2d

import numpy as np
import numpy.random as random
import scipy.sparse as sparse
from scipy.sparse.linalg import eigs, eigsh
import matplotlib.pyplot as plt

# number of sites
Nx, Ny = 100, 100

# hopping and randomness
t = 1.0
r = 0.1

# setup hamiltonian
N = Nx*Ny
H = sparse.lil_matrix((N,N))
H.setdiag(r*random.randn(N))

for ix in range(Nx):
    for iy in range(Ny):
        i = ix*Ny + iy
        H[i,(i+1) % N] = t
        H[i,i-1] = t
        H[i,(i+Ny) % N] = t
        H[i,i-Ny] = t
    ...
```

Pros

- Imperative, structured (make use of “functions”), modules
- Plenty of existing libraries and modules
- More than one way to write an algorithm
- Easy to learn!

Cons

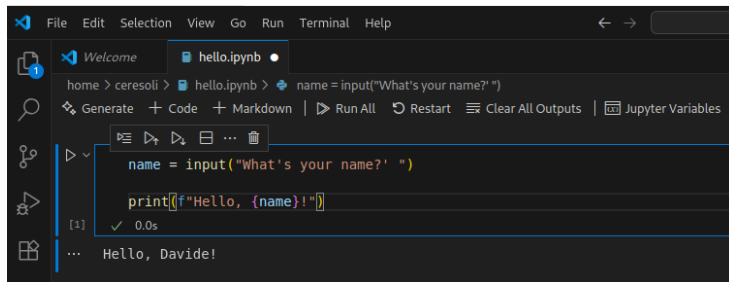
- Easy to mess up with data types
- Not CPU and memory efficient
- Easy to make mistakes!

Our first Python program (Linux shell)

```
ceresoli@bluestar:~$ cat hello.py
#!/usr/bin/env python

name = input("what's your name? ")
print(f"Hello, {name}")
ceresoli@bluestar:~$ chmod 0755 hello.py
ceresoli@bluestar:~$ ./hello.py
what's your name? Davide
Hello, Davide
ceresoli@bluestar:~$
```

Our first Python program (VSCode)



The screenshot shows the Visual Studio Code interface with a Jupyter notebook named 'hello.ipynb' open. The file explorer on the left shows the project structure: 'home > ceresoli > hello.ipynb'. The code editor displays the following Python code:

```
name = input("What's your name? ")  
  
print(f"Hello, {name}!")
```

The code has been executed, and the output is visible in the console at the bottom: 'Hello, Davide!'. The status bar at the bottom of the console shows '[1] ✓ 0.0s'. The top menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. The left sidebar contains icons for Explorer, Search, Source Control, Run and Debug, and Extensions.

Our first Python program (Jupyter)

jupyter hello (unsaved changes)

File Edit View Insert Cell Kernel Help

File Edit View Insert Cell Kernel Help

Run

```
In [1]: name = input("What's your name?' ")
        print(f"Hello, {name}!")
```

What's your name?' Davide
Hello, Davide!

In []:

- Low-level vs high-level languages
- Compilers vs interpreters
- The first program in Python
- Interactive coding with Jupyter notebooks

Questions?

Time to install and setup Python!

Next step: download lecture #1 notebook from MyAriel or from
<https://github.com/dceresoli/2026-Programming>.